

Score-Based Diffusion Model

その導入と基礎

Yukinari Hisaki

Contents

1. なぜ拡散モデルなのか？
2. スコアマッチングの基礎
3. 拡散の導入
4. ステップ 1: SDE の具体的な設計 (VP SDE)
5. ステップ 2: スコアの学習
6. ステップ 3: 生成 (Reverse ODE)
7. まとめ: 拡散フレームワーク
8. 制御可能な生成: SDE の Guidance

- 9. ODE が可能にする高度な機能
- 10. 生成の高速化: DPM-Solver++

なぜ拡散モデルなのか？

- **The Goal:** 機械学習を用いて、複雑な人間の運転行動をどのようにモデル化すべきか？
 - **Simple Approach (Regression):** 最も単純な方法は、コンテキスト c を単一の未来の軌跡 τ にマッピングする決定論的関数 $\tau = f_{\theta}(c)$ を学習することである。
 - ▶ 入力 c : 自車状態、他車、地図データなど。
 - ▶ 出力 τ : 状態系列 $\{x_t, x_{t+1}, \dots, x_T\}$ 。
 - **Limitations:** しかし、人間の行動は本質的に**マルチモーダル(多峰性)**であり、不確実である。
 - ▶ 曖昧な状況下では、有効な取るべき行動が複数存在する。
- ⇒ 単一のパスだけでなく、完全な分布 $p(\tau \mid c)$ を学習する**生成モデル**が必要である。

目標は、サンプル $x \sim q(x)$ から未知のデータ分布 $q(x)$ を推定することである。

- **Standard Approach (e.g., VAE):** 潜在変数 $z \sim \mathcal{N}(0, I)$ を導入し、**単一**の Decoder $p_\theta(x|z)$ を介してデータにマッピングする。

$$z \xrightarrow{p_\theta(x|z)} x$$

- **Limitations: Single shot** の $p_\theta(x|z)$ を介して複雑なデータ分布 $q(x)$ を学習することは困難である。単純な事前分布と複雑なデータ多様体間のギャップが大きすぎるためである。
- **Solution (Diffusion Models):** 生成を**複数の段階的な変換**に分解する。サンプルを step-by-step で改善していく。 $x_T \longrightarrow x_{T-1} \longrightarrow \dots \longrightarrow x_0$

$$x_T \sim \mathcal{N}(\cdot \mid 0, I), x_0 \sim q(x_0)$$

「拡散モデル」のアイデアは、数学的に主に3つの方法で定式化できる:

1. Denoising Diffusion Probabilistic Models (DDPM) [1], [2]

- 離散マルコフ連鎖として定式化。
- 変分下界 (ELBO) を介して最適化。
- キーアイデア: ノイズ付加をステップバイステップで逆転させることを学習する。

2. Score Matching [3]

- 確率微分方程式 (SDE) を介して定式化。
- キーアイデア: **スコア関数** $\nabla_x \log p_t(x)$ (より高い密度へ方向)を学習する。

3. Flow Matching [4]

- 常微分方程式 (ODE) に基づく、より一般的なフレームワーク。
- キーアイデア: 確率パスを輸送する**ベクトル場** $v_t(x)$ を直接回帰する。

Today's Focus: ここでは **2. Score Matching** に焦点を当てる。

スコアマッチングの基礎

目標: 未知の分布 $q(x)$ からのサンプルデータセット $\{x_1, \dots, x_N\}$ が与えられたとき、それを近似するモデル $p_\theta(x)$ を学習したい。

- **モデリングの課題:** 柔軟な確率モデルは、しばしば正規化定数(分配関数)を除いた形で定義される。(Energy-Based Models):

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}, \quad Z_\theta = \int \exp(-E_\theta(x)) dx$$

- **計算不能な分配関数:** 最尤推定 (MLE) で学習するには、 $\log p_\theta(x)$ を最大化する必要がある。しかし、 Z_θ (高次元空間での積分)の計算は計算量的に不可能である。

$$\log p_\theta(x) = -E_\theta(x) - \underbrace{\log Z_\theta}_{\text{計算不能!}}$$

- **解決策(微分):** 入力 x に関して勾配をとると、定数 Z_θ は消える！

$$\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x) - \underbrace{\nabla_x \log Z_\theta}_0 = -\nabla_x E_\theta(x)$$

密度の代わりに「対数密度の勾配」をモデル化することで、分配関数の問題を完全に回避できる。

定義 0.1 (スコア関数): 確率密度 $p(x)$ に対して、スコアは以下のように定義される:

$$s(x) := \nabla_x \log p(x).$$

- **直感: ベクトル場**

- ▶ スコアは、**最急上昇**(より高いデータ密度)の方向を指すベクトル場である。
- ▶ 重要なのは、 Z_θ を計算することなく有効な学習ターゲットを作成できることだ。

- **目的関数(スコアマッチング):** ニューラルネットワーク $s_\theta(x)$ をデータのスコアに一致させるように学習する:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{x \sim q} [\| s_\theta(x) - \nabla_x \log q(x) \|_2^2]$$

問題: 真のスコア $\nabla_x \log q(x)$ は計算不能である。

定理 0.1 (明示的スコアマッチングとデノイジング・スコアマッチングの等価性): $q(x)$ をデータ分布、 $q(\tilde{x}|x)$ を摂動カーネルとする。平滑化された周辺分布を $q(\tilde{x}) = \int q(\tilde{x}|x)q(x) dx$ と定義する。平滑化周辺分布に対する明示的スコアマッチング目的関数

$$\mathcal{L}_{\text{ESM}}(\theta) = \mathbb{E}_{q(\tilde{x})} [\| s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q(\tilde{x}) \|_2^2],$$

を最適化することは、 θ に依存しない定数 C を除いて、以下のデノイジング・スコアマッチング目的関数を最適化することと等価である:

$$\mathcal{L}_{\text{DSM}}(\theta) = \mathbb{E}_{q(x)q(\tilde{x}|x)} [\| s_{\theta}(\tilde{x}) - \nabla_{\tilde{x}} \log q(\tilde{x}|x) \|_2^2].$$

$$\Rightarrow \mathcal{L}_{\text{ESM}}(\theta) = \mathcal{L}_{\text{DSM}}(\theta) + C$$

Core Intuition: 周辺スコア $\nabla_{\tilde{x}} \log q(\tilde{x})$ の計算は、 $q(\tilde{x}) = \int q(\tilde{x}|x)q(x) dx$ が未知のデータ分布 $q(x)$ に依存するため計算不能である。この定理により、 $q(\tilde{x}|x)$ に依存する $\mathcal{L}_{\text{DSM}}$ を代わりに使用できるようになる。 **重要な点として**、摂動カーネル $q(\tilde{x}|x)$ は未知の確率分布 $q(x)$ から独立している。このカーネルは我々自身が設計するため、そのスコア $\nabla_{\tilde{x}} \log q(\tilde{x}|x)$ は解析的に計算可能である。 **条件付き分布の選択:** 典型的には、 $q(\tilde{x}|x)$ として **分散の小さいガウス分布** を選択する(例: $\mathcal{N}(\tilde{x}; x, \sigma^2 I)$)。これにより、条件付きスコアは解析的に扱いやすくなり、

$$\nabla_{\tilde{x}} \log q(\tilde{x}|x) = -\frac{\tilde{x} - x}{\sigma^2}$$

$q(\tilde{x})$ はデータ分布 $q(x)$ を密接に近似できるようになる。

スコア関数 $s_\theta(x) \approx \nabla_x \log q(x)$ を手に入れた後、どのようにサンプルを生成するか？

- **Langevin Dynamics:** x_0 を任意に初期化し、スコアを使用して反復的に更新する:

$$x_{t+1} \leftarrow x_t + \underbrace{\varepsilon_t s_\theta(x_t)}_{\text{勾配上昇}} + \underbrace{\sqrt{2\varepsilon_t} z_t}_{\text{ランダムノイズ}}$$

ここで、 $z_t \sim \mathcal{N}(0, I)$ であり、 ε はステップサイズである。

- **メカニズム:**

1. **勾配項:** x を高密度領域(モード)へ押し上げる。
2. **ノイズ項:** 単一のモードへの崩壊を防ぎ、分布全体の探索を可能にする。

$\Rightarrow t \rightarrow \infty$ かつ $\varepsilon_t \rightarrow 0$ とすると、 x_t は $q(x)$ からのサンプルに収束する。

理論は健全だが、このアイデアをナイーブに用いた学習は実際にはしばしば失敗する。

1. 多様体仮説 (The Manifold Hypothesis)

- 高次元データは低次元の多様体上に集中している。
- この多様体の外側では、 $q(x) \approx 0$ である。
- スコア $\nabla_x \log q(x)$ は**未定義**か、数値的に不安定である。

2. 低密度領域での不正確な推定

- 損失 $\mathbb{E}_{x \sim q}$ は、**データが存在する場所**でのみ誤差を最小化する。
- 低密度領域(空間の大部分)では、モデル $s_\theta(x)$ は学習されない。
- **結果:** ランダムノイズ(低密度)からサンプリングを開始すると、勾配が不正確になり、ゴミのような生成結果になる。

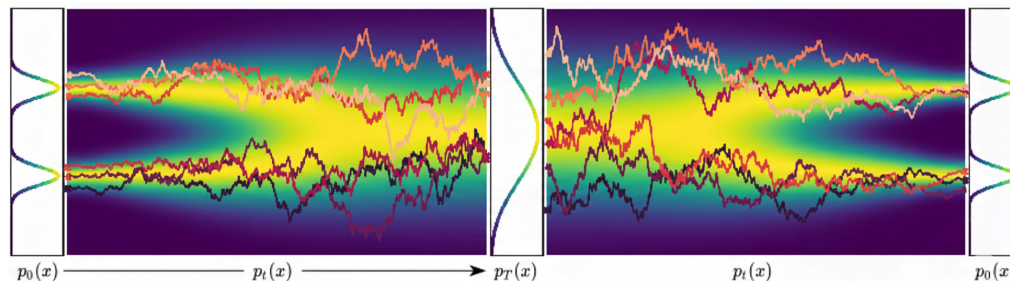
3. Langevin Dynamics の遅い混合 (Slow Mixing)

- データ分布が互いに素なモード(孤立したデータの「島」)を持つ場合、その間のスコアはほぼゼロになる。
- サンプラーは1つのモードにスタックし、他のモードへのギャップを越えられなくなる。

拡散の導入

確率が定義されている領域でスコアを計算することを避けるにはどうすればよいか？

- **Strategy: 連続的な分布の変形** q を直接モデル化する代わりに、以下をつなぐ $t \in [0, T]$ に対する分布の変形 p_t を考える:
 - ▶ $p_0 \approx q$ (複雑なデータ分布)。
 - ▶ $p_T \approx \mathcal{N}(0, I)$ (どこでも定義される扱いやすいノイズ分布)。
- **2つの重要な課題:**
 1. **学習:** p_t 自体が未知の場合、どのようにして時間依存スコア $s_t(x) = \nabla_x \log p_t(x)$ を学習するか？
 2. **サンプリング:** スコアが未定義の領域に足を踏み入れずに、どのようにサンプルを生成するか？



データ分布 p_0 は未知であるため、中間分布 p_t を数学的に直接定義することはできない。代わりに、各データ点を時間発展させる**プロセス**を定義する。

- **The Forward SDE:** データ点を徐々にノイズに変換する拡散過程を構築する:

$$dx = \underbrace{f(x, t) dt}_{\text{ドリフト: 決定論的变化}} + \underbrace{g(t) dw}_{\text{拡散: ランダムノイズ}}, \quad x_0 \sim p_0(x_0)$$

- **Forward SDE の設計:** 以下の 2 つの重要な条件を満たすように SDE を設計する必要がある:

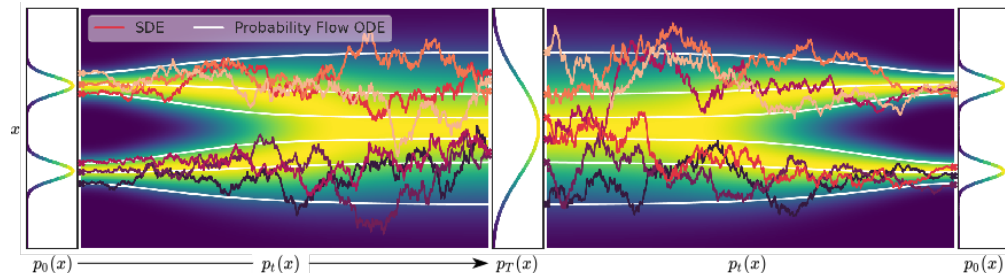
1. **計算可能な遷移:** 遷移確率 $p_{t|0}(x_t | x_0)$ は扱いやすいガウス分布でなければならない。これにより、学習中に SDE を step-by-step でシミュレーションする必要がなくなる。
2. **単純な事前分布:** プロセスは標準ガウス分布 $p_T(x) \approx \mathcal{N}(0, I)$ に収束し、サンプリングの既知の開始点を提供しなければならない。

⇒ 条件 1 により $p_{t|0}(x_t | x_0)$ が得られるため、未知の $p_t(x)$ を**デノイジング・スコアマッチング**を使用してスコアネットワーク $s_\theta(x, t)$ を学習できる。

すべてのタイムステップのスコア $s_\theta(x, t)$ が学習された後、どのようにサンプリングするか？ 単純な Langevin Dynamics では、スコアが不正確な低密度領域に踏み込んでしまう可能性がある。

- **確率フロー ODE (Probability Flow ODE):** すべての SDE に対して、その周辺分布 $p_t(x)$ がすべての t において SDE のものと等しい決定論的常微分方程式 (ODE) が存在する。(後ほど詳しく説明)
- **サンプリングのメカニズム:**
 1. サンプル $x_T \sim p_T$ (単純なガウス分布)。
 2. 時間を T から 0 へ逆方向に ODE を解く。

⇒ 明確に定義された領域から始まる有効な確率フローに従うため、軌跡 x_t は常に スコアが定義される領域に留まり、未定義のスコア領域を回避する。



ここからは、3つの具体的なステップで実際のモデルを構築する:

1. **ステップ 1: SDE の設計 (VP SDE)** ガウスノイズへの収束を保証するために、特定のドリフト $f(x, t)$ と拡散 $g(t)$ を定義する。
2. **ステップ 2: スコアの学習** 勾配場 $\nabla_x \log p_{t(x)}$ を推定するためにニューラルネットワーク $s_\theta(x, t)$ を学習する。
3. **ステップ 3: 生成 (Reverse ODE)** 学習したスコアを確率フロー ODE に代入し、ノイズからデータへ逆方向に解く。

ステップ 1: SDE の具体的な設計 (VP SDE)

確率フローの標準的な選択肢は **Variance Preserving (VP) SDE** であり、これは DDPM の連続時間極限である。

- **SDE の定式化:** 時間依存ノイズスケジュール $\beta(t) > 0$ を用いてドリフトと拡散を定義する:

$$dx = \underbrace{-\frac{1}{2}\beta(t)x}_{\text{ドリフト: 信号を縮小}} dt + \underbrace{\sqrt{\beta(t)}}_{\text{拡散: ノイズを注入}} dw$$

- **なぜ「分散保存 (Variance Preserving)」なのか？** 初期データ x_0 が分散1を持つ場合、 x_t の分散はすべての t に対して 1 のままである。ドリフトはデータを原点に向かって滑らかに縮小し、拡散はガウスノイズを追加することでそれを完全に補償する。
- **計算可能な遷移カーネル:** この SDE は線形であるため、 x_0 から x_t への遷移確率は常に扱いやすいガウス分布となる。 $\alpha_t = \exp\left(-\int_0^t \beta(s) ds\right)$ とすると:

$$p_{t|0}(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$$

- **実際の実装(線形スケジュール):** 実際には、 $t \in [0, 1]$ とし、 $\beta(t)$ を線形関数として定義する:

$$\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min})$$

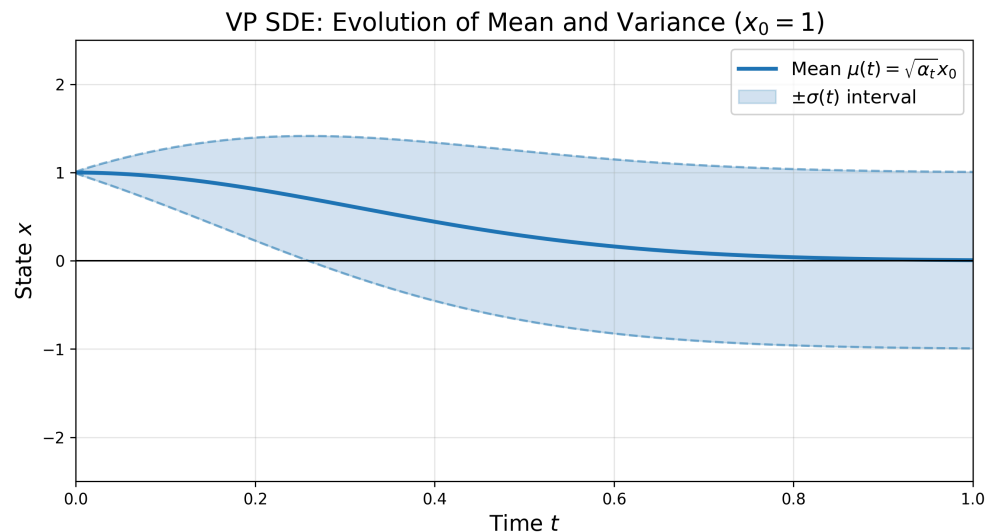
標準的なハイパーパラメータは $\beta_{\min} = 0.1$ および $\beta_{\max} = 20$ である。

$$\int_0^t \beta(s) \, ds = \beta_{\min} t + \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2$$

$$\Rightarrow \alpha_t = \exp\left(-\beta_{\min} t - \frac{1}{2}(\beta_{\max} - \beta_{\min})t^2\right)$$

$\Rightarrow t = 1$ において、 $\alpha_1 \approx \exp(-10) \approx 4.5 \times 10^{-5} \approx 0$ となる。信号は完全に破壊され、 $p_1(x) \approx \mathcal{N}(0, I)$ が保証される。

- **時間発展の可視化 ($x_0 = 1$):** これらの方程式はデータをどのように変換するか？ $x_0 = 1$ から始まる単一の 1 次元データポイントを時間 $t \in [0, 1]$ にわたって追跡する。



- **ドリフト (平均):** 平均 $\mu(t) = \sqrt{\alpha_t} x_0$ は 1 から 0 へ滑らかに減衰する。元の信号は体系的に消去される。
- **拡散 (分散):** $\pm\sigma(t)$ の区間(ここで $\sigma(t) = \sqrt{1 - \alpha_t}$)は、 $t = 1$ で分布が標準ガウスノイズ $\mathcal{N}(0, 1)$ に完全に一致するまで着実に拡大する。

ステップ 2: スコアの学習

具体的な SDE とその遷移カーネル $p_{t|0}(x_t|x_0)$ を設計できた。次は、すべての連続時間 $t \in [0, 1]$ にわたってスコア $\nabla_x \log p_t(x)$ を近似するようにニューラルネットワーク $s_\theta(x, t)$ を学習する。

- **デノイズング・スコアマッチングの拡張:** 先程は、計算不能な周辺スコア $q(x)$ に対して、それと同様に、計算不能な周辺スコアをマッチングさせることは、**既知の**条件付きスコア $\nabla_{x_t} \log p_{t|0}(x_t | x_0)$ をマッチングさせることと数学的に等価である。
- **VP SDE の条件付きスコア:** 遷移カーネルは $p_{t|0}(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$ であるため、スコアは単純にガウス対数密度の勾配となる:

$$\nabla_{x_t} \log p_{t|0}(x_t | x_0) = -\frac{x_t - \sqrt{\alpha_t}x_0}{1 - \alpha_t}$$

- **連続時間の目的関数:** この条件付きスコアへの期待距離を最小化することで、スコアネットワーク $s_\theta(x_t, t)$ を学習する。時間 t を一様に、元データ x_0 、および摂動データ x_t をサンプリングする。

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(0,1)} \mathbb{E}_{x_0 \sim p_0} \mathbb{E}_{x_t \sim p_{t|0}} \left[\lambda(t) \left\| s_{\theta}(x_t, t) - \underbrace{\left(-\frac{x_t - \sqrt{\alpha_t} x_0}{1 - \alpha_t} \right)}_{\nabla_{x_t} \log p_{t|0}(x_t | x_0)} \right\|_2^2 \right]$$

- **重み付け関数 $\lambda(t)$:** スコアの大きさは $t \rightarrow 0$ につれて極めて大きくなる(分散 $(1 - \alpha_t) \rightarrow 0$ となるため)。勾配爆発を防ぎ、すべての時間で損失のバランスをとるために、典型的には分散に比例する正の重み付け関数、例えば $\lambda(t) = 1 - \alpha_t$ を選択する。

⇒ この単一の目的関数を最適化することで、 $s_\theta(x, t)$ は連続ベクトル場を学習することが出来る。

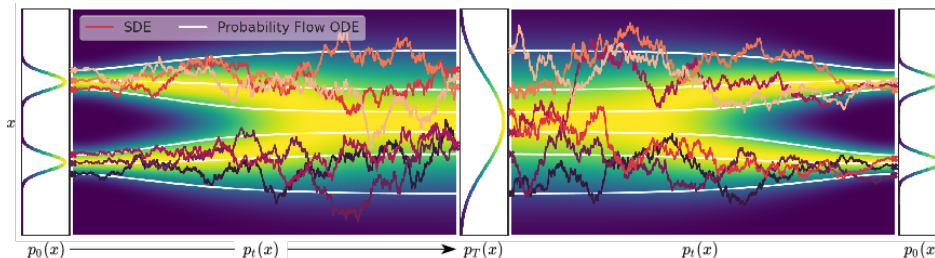
ステップ 3: 生成 (Reverse ODE)

データを生成するには、拡散過程を逆転させる必要がある。しかし、SDE の符号を反転させるだけでは逆過程を得ることは出来ない。そこで、確率過程 (SDE) と決定論的過程 (ODE) を結びつける基本定理を利用する。

定理 0.2 (確率フロー ODE [3]): 任意の SDE $dx = f(x, t) dt + g(t) dw$ に対して、対応する決定論的な常微分方程式 (ODE) が存在する。この ODE は、すべての時間 t において SDE と全く同じ周辺確率密度 $p_t(x)$ を共有する:

$$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt$$

- **含意:** もしスコア $\nabla_x \log p_t(x)$ がわかれば、ノイズ分布 p_T からデータ分布 p_0 へ決定論的にサンプルを輸送できる。



実際には、未知の真のスコアを学習済みニューラルネットワーク $s_\theta(x, t)$ に置き換える。

- **Generation:** VP SDE の項($f = -\frac{1}{2}\beta(t)x$, $g = \sqrt{\beta(t)}$)を代入し、以下の方程式を $t = T$ から 0 へ**逆方向**に解く:

$$dx = \left[-\frac{1}{2}\beta(t)x - \frac{1}{2}\beta(t)s_\theta(x, t) \right] dt$$

- **サンプリングステップ:**

1. **初期化:** 純粋なノイズ $x_T \sim \mathcal{N}(0, I)$ をサンプリングする。
2. **解く:** 数値解法(例: Euler、Runge-Kutta、または DPM-Solver)を使用して、 T から 0 まで ODE を積分する。
3. **出力:** 最終状態 x_0 が生成されたデータとなる。

まとめ: 拡散フレームワーク

確率フロー ODE に基づく生成モデルを構築した。

1. **順方向プロセス (VP SDE):** データを滑らかにガウスノイズに劣化させる線形 SDE を設計し、扱いやすい遷移カーネル $p_{t|0}(x_t | x_0)$ を提供した。
2. **学習 (スコアマッチング):** 解析的な条件付きスコアとマッチングさせることで、計算不能な周辺尤度をバイパスしてスコア関数 $s_\theta(x, t)$ を学習した。
3. **生成 (Reverse ODE):** 学習したスコアを使用してデータ点 $x_0 \sim p_T \approx q$ を正規分布 $x_T \sim p_T$ へ輸送する決定論的 ODE を逆方向に解くことで新しいデータのサンプルを可能にした。

制御可能な生成: SDE の Guidance

これまでの説明で無条件のデータ $x \sim p_0(x)$ を生成する方法は明らかになった。もしクラス y (例: テキストプロンプトやクラスラベル) を条件としてデータを生成したい場合はどうするか？

- **条件付きスコア:** 条件付きサンプルを生成するには、Reverse ODE 内の無条件スコア $\nabla_x \log p_t(x)$ を条件付きスコア $\nabla_x \log p_t(x | y)$ に置き換えるだけでよい。
- **ベイズの定理の適用:** 事後確率 $p_t(x | y) \propto p_t(x)p_t(y | x)$ の勾配をとることで、以下のような分解が得られる:

$$\nabla_x \log p_t(x | y) = \underbrace{\nabla_x \log p_t(x)}_{\substack{\text{unconditional score} \\ s_\theta(x, t)}} + \underbrace{\nabla_x \log p_t(y | x)}_{\substack{\text{Guidance 項} \\ (\text{尤度})}}$$

- **メカニズム:**
 1. **無条件スコア:** 生成されたサンプルが(データから学習した)リアルで高品質なデータに見えることを保証する。
 2. **Guidance 項:** ベクトル場として機能し、軌跡を条件 y を満たすデータ多様体の領域へと押しこむ。

Diffusion Planner では、特定の制約(例: 障害物回避、目標速度)を満たす有効な軌跡を生成したい。

- **Rethinking the Guidance Term:** $p_t(y \mid x)$ を予測するために別のニューラルネットワーク分類器を学習する代わりに、これらのルールを**人間が設計したコスト関数** $\mathcal{E}(x)$ を用いて数学的に定義できる。
⇒ 制約を満たす軌道の確率を高く，満たさない軌道の確率を低くするようなスコアを定義できる。
- **Energy-Based Formulation:** 条件 y をボルツマン分布として定式化する。軌跡が有効である確率は、コストが増加するにつれて指数関数的に減少する:

$$p_t(y \mid x_t) \propto \exp(-\lambda \mathcal{E}(x_t))$$

ここで、 $\lambda > 0$ は Guidance の強さを制御し、 $\mathcal{E}(x_t)$ は総コストである。

- **Relationship between Trajectory Optimization and Guidance:** $\mathcal{E}(x_t)$ は一般的な軌道最適化のコスト関数として捉えることができる。Guidance を適用した生成は、学習による生成と軌道の最適化の両方を組み合わせたものと解釈することができる。
- **Analytic Guidance Field:**

$$\nabla_{x_t} \log p_t(x_t | y) = s_\theta(x_t, t) - \lambda \nabla_{x_t} \mathcal{E}(x_t)$$

(例: $\mathcal{E}(x_t) = \mathcal{E}_{\text{obstacle}}(x_t) + \mathcal{E}_{\text{velocity}}(x_t)$)

⇒ 無条件スコア $s_\theta(x_t, t)$ は、人間の運転行動を模倣し、一方で $-\lambda \nabla_{x_t} \mathcal{E}(x_t)$ は推論中にパスを設計された制約に沿う方向への「斥力」として機能する。

ODE が可能にする高度な機能

確率フロー ODE は完全に**決定論的**であるため、複雑なデータ空間 $x_0 \sim p_0$ と単純な潜在空間 $x_T \sim p_T$ の間に全単射(1 対 1)写像を定義する。

- **補間の方法: 画像の補間の例**

1. **Encode:** 2 つの実画像 $x_0^{(A)}$ と $x_0^{(B)}$ を用意する。

ODE を**順方向**にシミュレーションし ($t = 0 \rightarrow T$)、それらの正確な潜在表現 $x_T^{(A)}$ と $x_T^{(B)}$ を見つける。

2. **補間:** 潜在空間でパラメータ $\lambda \in [0, 1]$ を使用して潜在変数を作成する(例: $z_\lambda = \text{Slerp}(x_T^{(A)}, x_T^{(B)}, \lambda)$)。

3. **デコード:** z_λ から開始して ODE を**逆方向**にシミュレーションし ($t = T \rightarrow 0$)、中間画像 $x_0^{(\lambda)}$ を生成する。

- **応用方法:** たとえば、経路生成の文脈で潜在空間を活用すると、異なるプランナーから出力される経路を潜在空間上でなめらかに補間し、経路の急な変化を防ぐというような応用が考えられる。



Reverse ODE により、任意のデータ点の**正確な**確率密度 $p_0(x)$ を計算できる。

- **瞬時変数変換公式:** $f_{\text{ODE}}(x, t)$ を確率フロー ODE のベクトル場とする。対数確率の時間変化は、このベクトル場の負の発散(ダイバージェンス)によって与えられる:

$$\frac{\partial \log p_t(x)}{\partial t} = -\nabla_x \cdot f_{\text{ODE}}(x, t)$$

- **尤度の計算:** 実画像 x_0 の正確な対数尤度を求めるには、 $t = 0$ から $t = T$ まで ODE 軌跡に沿ってこの発散を積分する:

$$\log p_0(x_0) = \log p_T(x_T) + \int_0^T \nabla_{x_t} \cdot f_{\text{ODE}}(x_t, t) dt$$

ここで、 $p_T(x_T)$ は単純なガウス事前分布 $\mathcal{N}(0, I)$ である。

- **応用方法:** 尤度を用いることによって、モデルの評価や異常検知が可能になる。

生成の高速化: DPM-Solver++

確率フロー ODE は、標準的な数値解法(Euler や Runge-Kutta など)で解くのは非常に遅く、多くの場合 200~1000 ステップを必要とする。

- **なぜそんなに遅いのか？** ODE には「剛性 (stiff)」のある線形ドリフト項が含まれている。大きなステップサイズをとると、打ち切り誤差が爆発し、画像が破壊される。

$$dx = \underbrace{-\frac{1}{2}\beta(t)x}_{\text{線形ドリフト (剛性あり)}} dt - \frac{1}{2}\beta(t) \underbrace{\varepsilon_{\theta}(x, t)}_{\text{ニューラルネットワーク}} dt$$

- **半解析的解法(指数積分器)**: ODE 全体をブラックボックスとして扱うのではなく、その半線形構造を利用する。定数変化法(variation of constants formula)を適用することで、線形部分 $-\frac{1}{2}\beta(t)x$ を**正確に解析的に**解くことができる。

$$x(t) = \underbrace{e^{-\int_s^t \frac{1}{2}\beta(r) dr} x(s)}_{\text{正確な線形発展}} - \int_s^t e^{-\int_u^t \frac{1}{2}\beta(r) dr} \frac{1}{2}\beta(u)\varepsilon_\theta(x_u, u) du$$

⇒ ニューラルネットワーク部分 ε_θ の積分のみを数値的に近似すればよい。 ε_θ (またはデータ予測 x_θ) は生のドリフトよりもはるかに滑らかに変化するため、最小限の誤差で巨大なステップをとることができる！

References

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>
- [2] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics,” in Proceedings of the 32nd International Conference on Machine Learning (ICML), 2015. [Online]. Available: <https://proceedings.mlr.press/v37/sohl-dickstein15.html>
- [3] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,”

in International Conference on Learning Representations (ICLR), 2021. [Online]. Available: <https://openreview.net/forum?id=PxTIG12RRHS>

- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” in International Conference on Learning Representations (ICLR), 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [5] P. Vincent, “A Connection Between Score Matching and Denoising Autoencoders,” Neural Computation, 2011, doi: [10.1162/NECO_a_00142](https://doi.org/10.1162/NECO_a_00142).
- [6] Y. Song and S. Ermon, “Generative Modeling by Estimating Gradients of the Data Distribution,” in Advances in Neural Information Processing Systems (NeurIPS), 2019. [Online]. Available: <https://papers.nips.cc/paper/9361-generative-modeling-by-estimating-gradients-of-the-data-distribution>